

Analysis of Web Application Vulnerabilities using Dynamic Application Security Testing

Ravinder Singh,
Department of Computer Engineering,
Suresh Gyan Vihar University,
Jaipur, India
singhravindersingh@gmail.com

Dr. Mukesh Kumar Gupta
Department of Electrical Engineering,
Suresh Gyan Vihar University,
Jaipur, India,
mkgupta72@gmail.com

Dr. Dipak Raghunath Patil
Department of Computer Engineering,
AVCOE,
Sangamner, India,
dipak.patil@avcoe.org

Dr. Sarang Maruti Patil,
Department of Computer Engineering,
SKN Sinhgad Institute of Technology &
Science,
Lonavala, India,
patilsm.sknsits@sinhgad.edu

Abstract—In the era of ubiquitous digital connectivity, securing software applications against cyber threats is paramount. Dynamic Application Security Testing (DAST) has emerged as a crucial methodology in the realm of cybersecurity, providing a dynamic, real-time approach to identify and mitigate vulnerabilities within software applications. This research paper delves into the significance and application of DAST in fortifying cybersecurity measures. DAST represents a departure from traditional static testing methods, which analyze source code or binaries, by assessing applications during runtime. The methodology involves the active, controlled probing of live applications through simulated cyber-attacks, including injection of malicious payloads and emulation of common exploit techniques. By dynamically assessing how applications respond to these simulated threats, DAST offers a real-world evaluation of security posture, facilitating the identification of potential vulnerabilities.

This research paper analyzed the Dynamic Application Security Testing (DAST) reports for different web applications over the past two years and provide a valuable resource for cybersecurity practitioners, researchers, and organizations striving to adopt effective strategies in safeguarding their digital assets against evolving cyber threats in the current environment.

Keywords—Application Security, OWASP Top 10, vulnerabilities.

I. INTRODUCTION

The significance of identifying vulnerabilities becomes pronounced when a web application is susceptible to compromise through exploitation. Given the continual emergence of new vulnerabilities, it is imperative to actively identify and analyze existing ones. This research paper concentrates on the analysis of web application vulnerabilities using the Dynamic Application Security Testing techniques.

A. Website:

A website is a digital space accessible via the internet, comprising interconnected web pages under a common domain. It presents information, media, or services using technologies like HTML. Websites range from static pages to dynamic platforms, serving diverse purposes such as information dissemination, commerce, social interaction, or

education. Users access these pages through web browsers, navigating content structured by links and multimedia elements. Websites can be personal, business-oriented, or institutional, contributing to the expansive landscape of the World Wide Web. They facilitate global connectivity, enabling the seamless exchange of data and communication in an increasingly digital world.

B. Website security:

Website security involves the implementation of measures and practices aimed at safeguarding a website against unauthorized access, data loss, and cyber-attacks. It involves the use of technologies, protocols, and best practices to protect the integrity, confidentiality and availability of the website. The common philosophy is “the best defense against cyber-attacks is a good offense” [1]. Website security in terms of CIA is as follows:

(a) Confidentiality: Ensuring confidentiality means protecting sensitive information from unauthorized access. Implementing encryption protocols, particularly through technologies like SSL/TLS, helps in securing data during transmission. Access controls, authentication mechanisms, and proper user permissions also play a crucial role in maintaining confidentiality, preventing unauthorized individuals from viewing or accessing sensitive data.

(b) Integrity: Integrity focuses on the accuracy and reliability of data. Website security measures must ensure that data is not tampered with or altered by unauthorized parties. Techniques such as checksums, digital signatures, and hashing algorithms are employed to detect and prevent data manipulation. Regular audits, monitoring, and version control mechanisms help maintain the integrity of website content and user data.

(c) Availability: Availability ensures that a website is accessible and operational when needed. DDoS (Distributed Denial of Service) protection, redundancy in server infrastructure, and load balancing are essential for ensuring that a website remains available even during times of heavy traffic or in the face of malicious attacks. Adequate backup and recovery plans also contribute to maintaining availability by minimizing downtime in the event of a security incident or system failure.

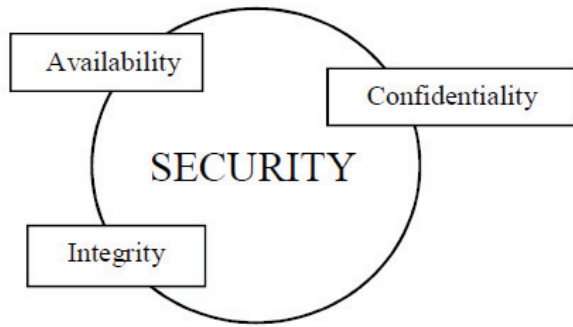


Fig. 1. Website Security Triad

By addressing confidentiality, integrity, and availability, website security strategies aim to create a robust defense framework that safeguards both the website and the sensitive data it handles.

C. Website vulnerabilities

Website vulnerabilities [2] are potential weaknesses in a site's security that can be exploited by attackers. Common vulnerabilities include SQL injection [3], where attackers manipulate databases, and cross-site scripting (XSS) [4], enabling unauthorized code execution in users' browsers. Inadequate authentication and authorization mechanisms may allow unauthorized access. Security misconfigurations, unpatched software, and outdated plugins pose risks. File inclusion vulnerabilities and improper error handling are also exploited. Regular security audits, updates, and adherence to secure coding practices are crucial to identify and mitigate vulnerabilities, ensuring a robust defense against cyber threats and preserving the integrity and confidentiality of the website and its data. Numerous web vulnerabilities, including CSRF, XSS, and SQLi are cataloged by the Open Web Application Security Project (OWASP) [5].

D. Types of Website vulnerabilities:

Web vulnerabilities encompass a huge range of weaknesses, flaws that possibly exploited by attackers to compromise the security of web applications and websites. Here are some common types of web vulnerabilities:

1. SQL Injection (SQLi): Malicious SQL code is injected by attackers into input fields with the aim of manipulating database queries.
2. Cross-Site Scripting (XSS): Malicious scripts are injected by attackers into web pages.
3. Cross-Site Request Forgery (CSRF): Attackers deceive authenticated users into executing unintended functions on a trusted site.
4. Security Misconfigurations: Insecure default settings, unnecessary services, or incomplete configurations.
5. Insecure Direct Object References (IDOR): Improper access control that allows attackers to reference and access internal objects directly.
6. Security Headers Missing: Lack of crucial security headers, such as Content Security Policy or Strict-Transport-Security.

7. File Inclusion Vulnerabilities: Improper input validation allows attackers to include files from a remote server.
8. Unvalidated Redirects and Forwards: Open redirects or forwards that can be exploited to trick users into visiting malicious sites.
9. Sensitive Data Exposure: Failure to properly protect sensitive information, such as passwords or personal data.
10. XML External Entity (XXE) Attacks: Exploitation of insecure processing of XML input to disclose internal files.

E. Dynamic Application Security Testing (DAST):

DAST is a way to identifying and mitigating security vulnerabilities within software applications. Unlike static testing methods, DAST simulates real-world attack scenarios by dynamically analyzing running applications. By sending controlled traffic and payloads, DAST evaluates how well an application withstands potential threats, such as XSS, SQLi and other attacks. This dynamic analysis ensures a comprehensive examination of the application's security, enabling organizations to detect and remediate vulnerabilities promptly.

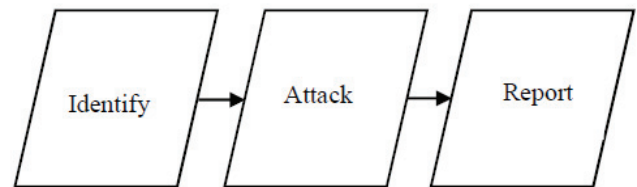


Fig. 2. DAST Flow

DAST plays a pivotal role in enhancing the overall cybersecurity posture, offering real-time insights into application vulnerabilities and empowering organizations to fortify their digital assets against evolving cyber threats. The significance of DAST lies in its ability to proactively discover and address security weaknesses, minimizing the risk of exploitation. By offering a dynamic and immediate assessment of an application's security, DAST empowers organizations to rapidly identify and remediate vulnerabilities, thereby enhancing overall cybersecurity. As digital landscapes evolve and cyber threats become more sophisticated, DAST serves as a crucial tool in the existing effort to protect applications and secure sensitive data from unauthorized access or manipulation. Its real-time insights contribute to a resilient cybersecurity strategy, allowing organizations to adapt and fortify their defenses against emerging threats in an ever-changing technological landscape.

II. RELATED WORK

Khalid et. al [6] have introduced a black box testing approach designed to identify various web vulnerabilities, including SQLi, XSS, and CSRF. Additionally, they created a detection tool named Web Vulnerabilities Finder to identify the majority of these vulnerabilities. This method enables automated website analysis to uncover web vulnerabilities. Upon testing the tool on several websites, it successfully identified 45 exploitable XSS, 45 SQLi, 38 Directory Disclosure, and 40 Local/Remote File Inclusion

vulnerabilities. The experimental findings demonstrate the tool's effectiveness in efficiently detecting the SQLi, XSS, Directory Disclosure, and LFI/RFI vulnerabilities.

The Web Application Vulnerability Scanners (WAVS) are used in the development area so that the security of website can be assess continuously, aiming to identify potential vulnerabilities that could pose a threat to client services. This research assesses the efficacy and precision of 5 WAVS tools i.e. Acunetix WVS, Burp Suite, NetSparker, Nessus, and OWASP ZAP in detecting potential vulnerabilities in websites. These scanners were chosen from the top ten recommended scanning software in Year 2017. The evaluation employed the black box testing method to assess the performance of the 5 WAVS tools across 7 vulnerable websites. Various metrics, such as the severity level of vulnerabilities, types of detected vulnerabilities, the count of false positive vulnerabilities, and the accuracy of each scanner, were used in the evaluation. The assessment was based on a compiled list of vulnerabilities extracted from OWASP and NIST, with accuracy gauged by the identification of true and false positives for each scanner. [7]

The web applications harbor various vulnerabilities, each susceptible to web attacks, with SQLi and XSS attacks being prominent methods. This approach primarily relies on SQLi Detection and XSS detection methods integrated with a crawling technique. Initially, users input a URL to check for vulnerabilities and initiate the scanning process by clicking 'Start.' Subsequently, the application commences crawling for SQLi and XSS vulnerabilities. If any vulnerabilities are identified, they are logged in the spider log. The detected vulnerabilities are then compiled into a report, providing users with a clear understanding of the application's weaknesses. [8]

Maini et. al [9] Designed a scanning tool to detect prevalent vulnerabilities in web applications, encompassing SQLi, XSS, CRLF Injection, and Open Redirect. The tool is equipped with a basic port scanner with web crawling facility to identify additional services potentially operating on the web server. The research introduces a straightforward black-box security testing technique to identify these issues. The paper concludes by demonstrating the simplicity of scanning a sophisticated enterprise-grade web application using the developed scanner. The primary objective of the scanning software is to expose vulnerabilities and generate a comprehensive and effective report for each website.

Mubaiwa et. al [10] Proposed an experiment focusing on an innovative tool employing a hybrid methodology that integrates both white-box and black-box testing practices. The development of the hybrid algorithm is not arbitrary; rather, it is meticulously crafted based on exceptional factors such as optimization and complexity, among others, to enhance overall effectiveness. This algorithm adeptly identifies SQLi and XSS and can be applied to any real-world application running on a web server, where interactions between the client and the database occur. The scanning process involves crawling and parsing to unveil vulnerabilities, which is iteratively performed until all vulnerabilities are identified. A prototype was implemented to assess and validate the efficacy of the hybrid approach.

III. METHODOLOGY

The methodology used in DAST is designed to check the security of websites in real-time by actively probing and analyzing the application during runtime.

Below are the key steps and components typically involved in the DAST methodology:

1. **Application Discovery:** Identify and enumerate the target application or web service. Establish a scope for testing, including specific URLs and parameters.
2. **Configuration and Setup:** Configure the DAST tool with relevant settings, including authentication credentials if required. Customize scan parameters based on the target application and testing goals.
3. **Crawling and Analysis:** DAST tools simulate a user interacting with the application, crawling through the application's pages and functionality. Identify inputs and outputs, forms, and parameters to understand the application's structure.

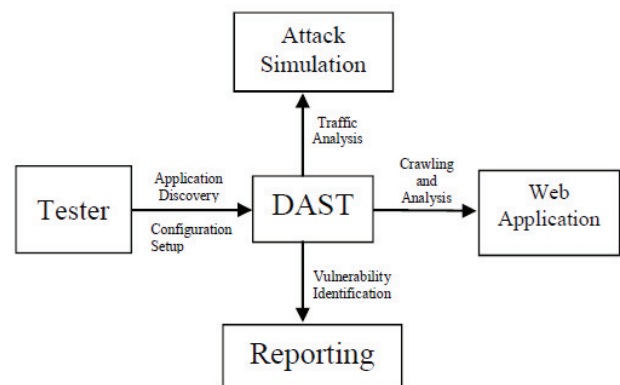


Fig. 3. DAST Methodology

4. **Attack Simulation:** DAST tools generate and send a variety of attack payloads to inputs, such as command injection, reflected XSS, blind SQLi etc. Simulate various attack scenarios to identify potential security weaknesses.
5. **Traffic Analysis:** Monitor and analyze the traffic between the DAST tool and website. Capture and inspect responses to identify anomalous or unexpected behavior
6. **Vulnerability Identification:** DAST tools analyze responses to identify vulnerabilities, including security misconfigurations, input validation issues, and other weaknesses. Severity wise listing of vulnerabilities.
7. **Reporting:** Produce detailed reports that provide information about the vulnerabilities. Also provide the remediation along with the severity level. Give remediation to the developers so that they can overcome the reported vulnerabilities.

In this research paper, we have analyzed the Dynamic Application Security Testing (DAST) reports generated by HCL Appscan Tool for different web applications over the past two years and provide a valuable resource for cybersecurity practitioners, researchers, and organizations, what are the most common vulnerabilities are being observed now a day within the web applications. The results are

beneficial for identifying and mitigating security vulnerabilities, prioritizing risks, ensuring compliance, fostering a security-aware culture, and ultimately improving the overall security posture of an organization's web applications.

IV. RESULTS AND DISCUSSION

The comprehensive research with detail identified vulnerabilities, their severity levels, and recommendations for remediation data for the 12 different user logins have been analyzed in this paper. The term 'Common Vulnerability Scoring System Score' represents the holistic security effect of a vulnerability, the total score takes into account metrics from three different categories: Base, Temporal, and Environmental. The score derived from available information, such as values, for one or more of these metrics. The CVSS is related with the severity level as per the following formula:

TABLE I. CVSS SCORE VS VULNERABILITY SEVERITY

Vulnerability Severity	CVSS Score
High	Between 7.5 - 10
Medium	Between 5.1 - 7.5
Low	Between 2.0 - 5.1

A high vulnerability signifies a critical security flaw that, if exploited, could result in severe consequences. This may include unauthorized access to sensitive systems, compromise of crucial data, or significant disruption of services. Such vulnerabilities demand urgent attention and

remediation measures to mitigate the risk of exploitation. The fig 4: High Severity Vulnerabilities and fig 7: Major Contributors in High Severity Vulnerabilities, indicating that the 'Blind Remote Command Execution through Bash (36%)', 'reflected

XSS(18%)' and 'Blind SQL Injection (12%)' are the major contributors with 156, 78 and 52 issues respectively.

Similarly, Medium vulnerabilities present a moderate security risk, with potential consequences such as compromised data or services. While they require timely attention, urgency is less than high vulnerabilities. The fig 5: Medium Severity Vulnerabilities, denoting that the CSRF is the most persistent vulnerability with the 204 issues.

Low vulnerabilities have minimal impact, demanding attention but are of lower priority, often addressed based on available resources and overall risk landscape. The fig 6: Low Severity Vulnerabilities, explaining that 'unsafe third party link', 'Host header injection' and 'database error pattern found' are most dominating vulnerabilities in low severity category with 225, 107 and 52 number of issues respectively.

The CVSS score serves as the determining factor for categorizing vulnerabilities into High, Medium, or Low severity levels. Consequently, Table 2: Common Vulnerabilities with CVSS score, providing details on those with high CVSS scores and those with low CVSS scores.

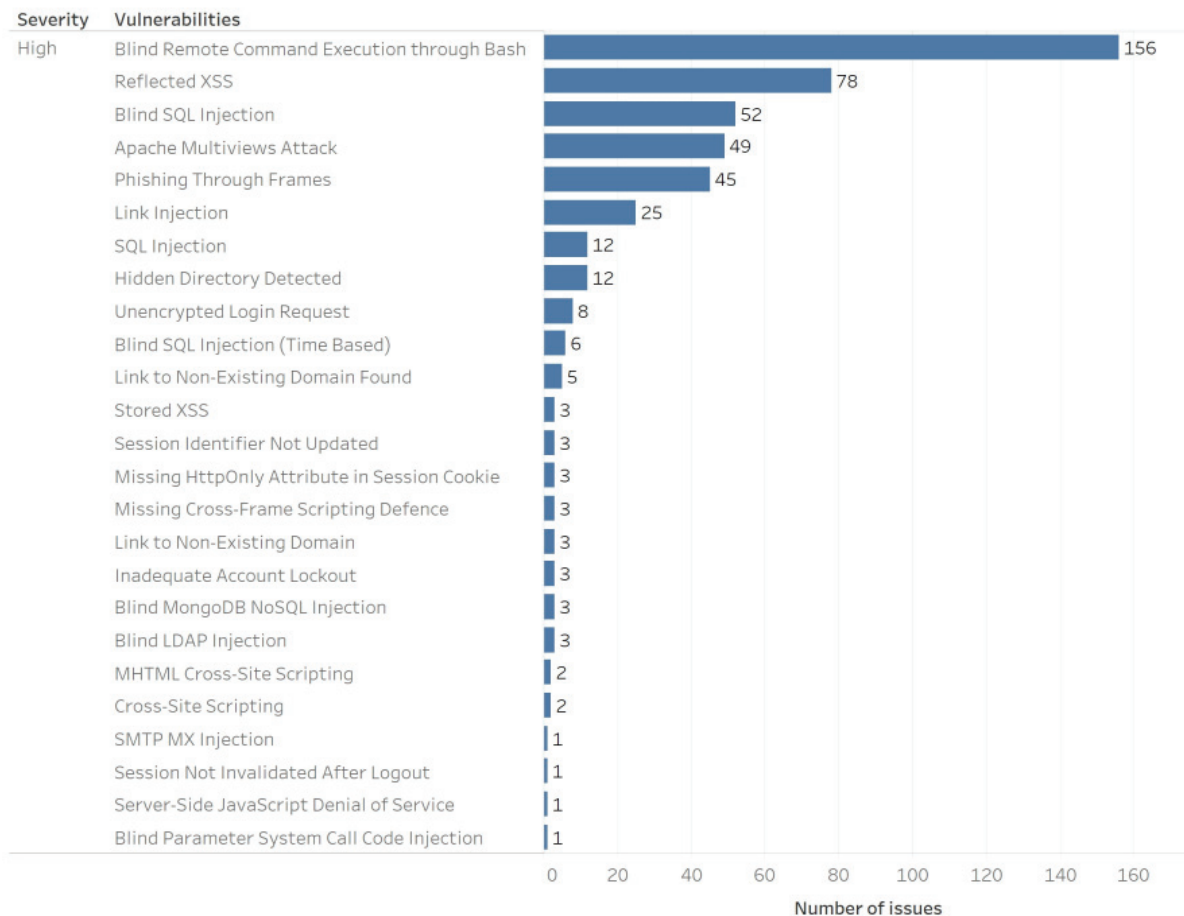


Fig. 4. High Severity Vulnerabilities

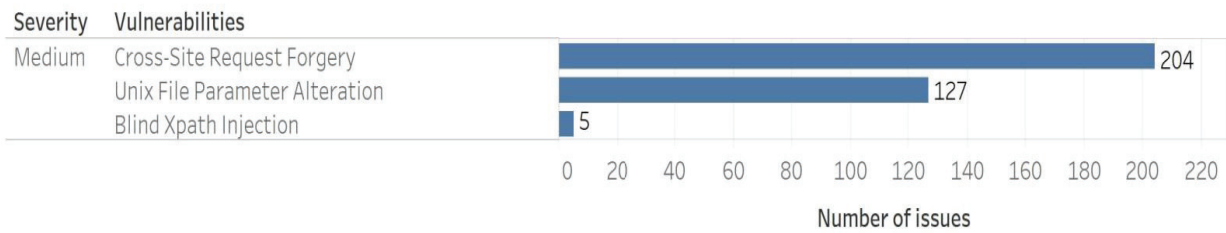


Fig. 5. Medium Severity Vulnerabilities

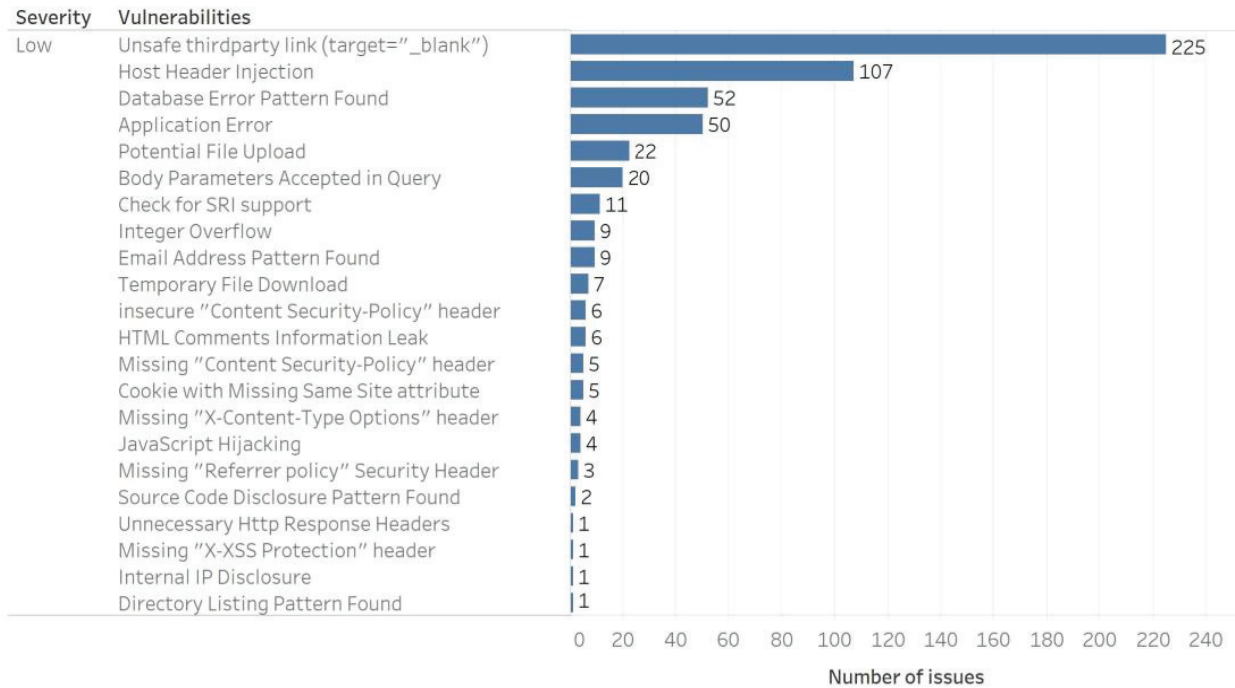


Fig. 6. Low Severity Vulnerabilities

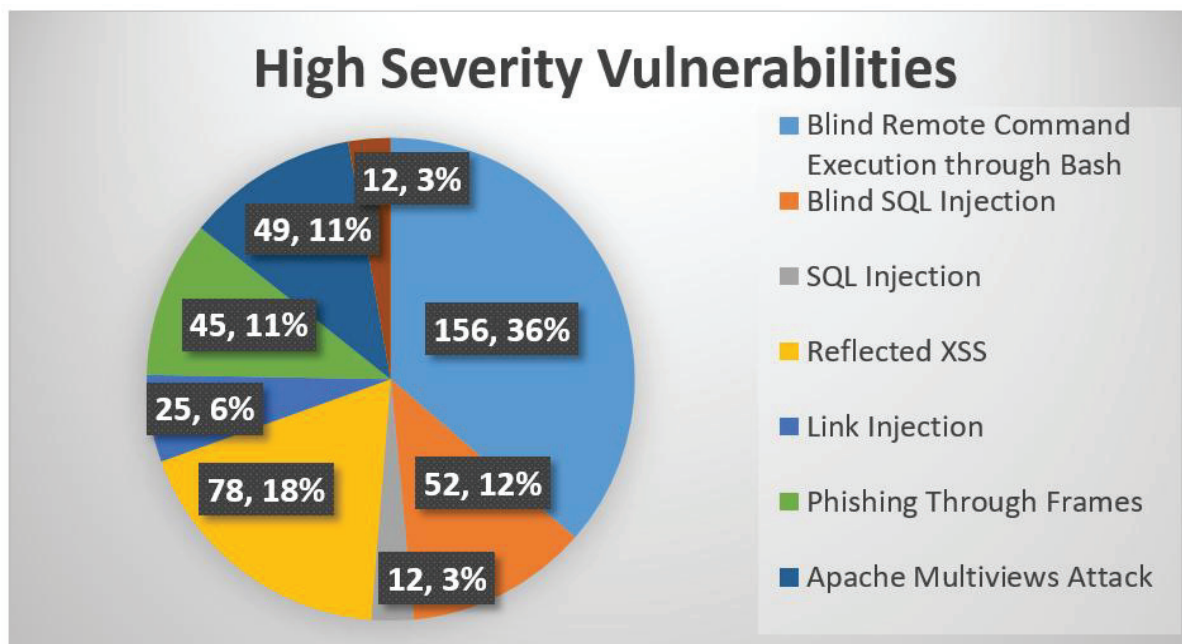


Fig. 7. Major Contributors in High Severity Vulnerabilities

TABLE II. COMMON VULNERABILITIES WITH CVSS SCORE

Vulnerability	CVSS Score
Blind MongoDB NoSQL Injection	10
Unencrypted Login Request	10
SQL Injection	9.7
Reflected XSS	9.4
Blind LDAP Injection	9
Phishing Through Frames	8.9
Unencrypted Login Request	8.5
MHTML Cross-Site Scripting	8.4
Apache Multiviews Attack	8
Cross-Site Scripting	7.5
Cross-Site Request Forgery	6.4
Missing "Content Security-Policy" header	5
Database Error Pattern Found	5
Cookie with Missing Same Site attribute	4.1

V. CONCLUSION

In the culmination of the assessment conducted, a comprehensive understanding of security landscape has been achieved. This examination aimed to proactively identify the vulnerabilities, then fortifying the application against potential cyber threats. Prioritizing based on severity, our assessment highlights critical vulnerabilities demanding immediate attention. This pragmatic categorization facilitates a focused and efficient remediation process, addressing the most impactful issues first. The analysis delivers a comprehensive evaluation of the web application's security, uncovering vulnerabilities from common issues to potential authentication weaknesses. Prioritizing these findings by severity provides a strategic guide for focused remediation efforts. Swiftly addressing identified vulnerabilities not only enhances overall security but also mitigates the risk of data breaches, safeguarding sensitive information. Beyond technical insights, the research fosters awareness among stakeholders, encouraging a proactive approach to security. It serves as a critical tool in the commitment to maintaining a resilient and secure web application environment.

This research is instrumental in providing a roadmap for securing web applications. By offering dynamic insights into the evolving threat landscape, it equips organizations with

the necessary tools to fortify their digital assets. The collaborative efforts outlined in the research, encompassing vulnerability remediation, continuous monitoring, and proactive security measures, contribute significantly to building a robust defense against cyber threats.

REFERENCES

- [1] Z. B. Babovic, J. Protic, and V. Milutinovic, "Web Performance Evaluation for Internet of Things Applications," *IEEE Access*, vol. 4, pp. 6974-6992, 2016
- [2] Awolaye, Olusesan M., Blessing Ojuloge, and Mathew O. Ilori. "Web application vulnerability assessment and policy direction towards a secure smart government." *Government Information Quarterly* 31 (2014): S118-S125
- [3] M.N.Kavitha, V. Vennila, G.Padmapriya, A. Rajiv Kannan Prevention Of Sql Injection Attack Using Unsupervised Machine Learning Approach" *International Journal of Aquatic Science* ISSN: 2008-8019 Vol 12, Issue 03, 2021
- [4] K. Joylin Bala, E.Babu Raj, A. M. Anusha Bimini. "XSS Attack Prevention over Code Injection Vulnerabilities in Web Applications" *International Journal of Innovative Technology and Exploring Engineering (IJITEE)* ISSN: 2278-3075, Volume-8 Issue-8 June, 2019
- [5] OWASP: Available at <https://owasp.org/www-project-top-ten>. OWASP Top Ten Project, 2021
- [6] Muhammad Noman Khalid, Muhammad Iqbal, Kamran Rasheed, Malik Muneeb Abid "Web Vulnerability Finder (WVF): Automated Black-Box Web Vulnerability Scanner" *I.J. Information Technology and Computer Science*, 2020, 4, 38-46
- [7] Malik Qasaimah, Ala'a Shamlawi, Tariq Khairallah, Black Box Evaluation Of Web Application Scanners: Standards Mapping Approach, *Journal Of Theoretical And Applied Information Technology*, 2018
- [8] Trupti Bhosale, Shraddha More, Prof. S.N. Mhatre, "Testing Web Application using Vulnerability Scan" *International Research Journal of Engineering and Technology (IRJET)* e-ISSN: 2395-0056 Volume: 06 Issue: 05 | May 2019
- [9] Rahul Maini, Rahul Pandey, Rajeev Kumar, Rajat Gupta, "Automated Web Vulnerability Scanner" *International Journal of Engineering Applied Sciences and Technology*, 2019 Vol. 4, Issue 1, ISSN No. 2455-2143, Pages 132-136
- [10] [10] Tsitsi G. Mubaiwa; M Mukosera, "A Hybrid Approach To Detect Security Vulnerabilities In Web Applications" *IJCSMC*, Vol. 11, Issue. 2, February 2022, pg.89 – 98